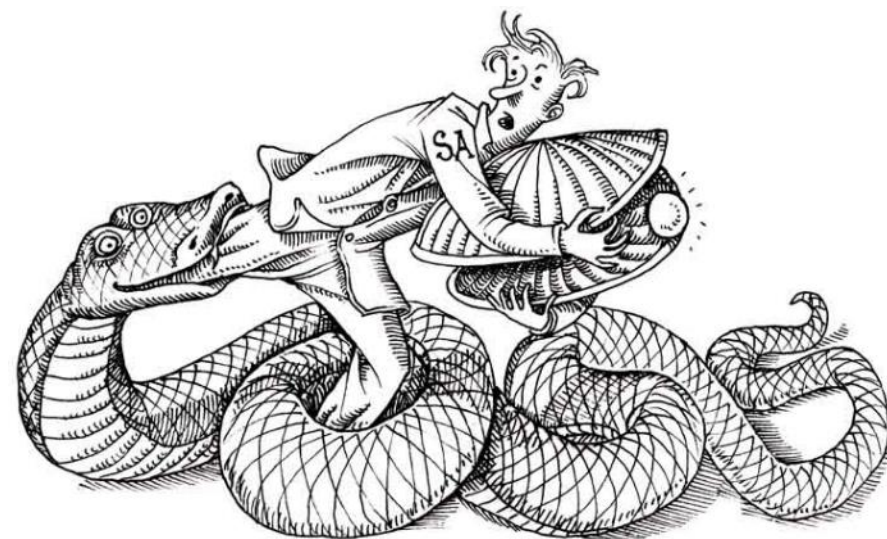


Shell Programming

Portable shell scripting, control flow, robustness, sed, and awk

tsaimh (2022-2026, CC BY-SA)
lctseng (2019-2021, CC BY-SA)
? (1996-2018)



What You Will Learn

- Write portable shell scripts using `/bin/sh` syntax and safe quoting.
- Use variables, positional parameters, tests, arithmetic, and control structures.
- Structure scripts with functions and `getopts`; understand exit status and signals.
- Debug scripts with tracing and `ShellCheck`, and clean up temporary resources safely.
- Use regular expressions, `sed`, and `awk` for practical text-processing pipelines.

Course emphasis Prefer portable, readable, and predictable shell code over clever one-liners.

Roadmap

Shell language

- Interpreter & shebang
- Variables & quoting
- Parameters & tests
- Arithmetic & control flow

Robust scripting

- Functions & options
- Exit status & signals
- Debugging & ShellCheck
- Regex, sed & awk

1. Foundations

What a shell script is, which shell we target, and how execution works.

Why Shell Programming?

- Automate repeated command-line tasks without writing a full application.
- Glue existing tools together with pipes, redirections, and exit status.
- Excellent for system administration, deployment, monitoring, and batch jobs.
- The shell is an interpreter: the script is read and executed as commands.

Rule of thumb Shell is strongest when most of the real work is done by existing commands.

Which Shell Are We Programming?

This course: POSIX-style sh

- Use `#!/bin/sh`
- Portable across Debian, FreeBSD, and many Unix-like systems
- Avoid shell-specific features unless explicitly introduced

Other common shells

- bash: many extensions and arrays
- zsh: interactive features and scripting extensions
- tcsh: historically common on FreeBSD; very different scripting syntax

Important `/bin/sh` is an interface, not necessarily the historical Bourne shell. Its implementation differs by system.

Your First Script

```
#!/bin/sh

for i in 1 2 3; do
    printf 'Hello world %s\n' "$i"
done
```

- The first line selects the interpreter.
- Quote variable expansions unless you intentionally want word splitting or globbing.
- `printf` is more predictable than `echo` for formatted output.

Shebang and Execution

```
$ chmod +x hello.sh
$ ./hello.sh
Hello world 1
Hello world 2
Hello world 3

$ sh hello.sh # explicit interpreter; executable bit not required
```

- `#!/bin/sh` is the portable default for system scripts.
- `#!/usr/bin/env bash` can locate Bash via PATH, but PATH then influences which interpreter runs.
- Use an absolute interpreter path when reproducibility matters.

2. Variables & Parameters

Quoting, command substitution, parameter expansion, and script arguments.

Shell Variables

Purpose	Syntax	Scope
Shell variable	name=value	Current shell
Environment variable	export name=value	Inherited by child processes
Read value	"\$name" or "\${name}"	Expansion at use time

```
export PAGER=/usr/bin/less
month=$(date +%m)
name='shell programming'
printf '%s\n' "$name"
```

Syntax No spaces around = in assignments: name=value, not name = value.

Quoting: Single, Double, or None?

Form	What happens?	Example
'text'	Everything is literal	echo '\$USER'
"text"	Parameter and command substitution still happen	echo "Hello \$USER"
unquoted	Word splitting and pathname expansion may occur	echo \$files

Best practice Quote expansions: "\$var", "\$(command)", and "\$@" unless splitting is intentional.

Command Substitution

```
now=$(date)
count=$(wc -l < /etc/passwd)
printf 'Now: %s\nUsers: %s\n' "$now" "$count"
```

- `$(...)` is preferred over legacy backticks because it is clearer and nests cleanly.
- Command substitution removes trailing newlines from the captured standard output.

Parameter Expansion: Defaults and Checks

Expansion	Meaning when var is unset or empty
<code>\${var:-default}</code>	Use default, but do not assign it
<code>\${var:=default}</code>	Assign default, then use it
<code>\${var:?message}</code>	Print message to stderr and abort this expansion
<code>\${var:+alternate}</code>	Use alternate only when var is set and non-empty

```
config=${CONFIG_FILE:-/etc/myapp.conf}  
: "${HOME:?HOME must be set}"
```

```
$ echo $config  
$ config=${CONFIG_FILE:-/etc/myapp.conf}  
$ echo $config  
/etc/myapp.conf
```

```
$ :  
$ : "${HOME:?HOME must be set}"  
$ : "${HOM:?HOM must be set}"  
bash: HOM: HOM must be set
```

Parameter Expansion: String Operations

Expansion	Result
<code>\${#var}</code>	String length
<code>\${var#pattern}</code>	Remove shortest matching prefix
<code>\${var##pattern}</code>	Remove longest matching prefix
<code>\${var%pattern}</code>	Remove shortest matching suffix
<code>\${var%%pattern}</code>	Remove longest matching suffix

```
path=/var/log/system.log
base=${path##*/}          # system.log
stem=${base%.log}         # system
```

Positional Parameters

Parameter	Meaning
\$0	Script name
\$1, \$2, ...	Positional arguments
\$#	Number of positional arguments
\$?	Exit status of the most recent command
\$\$	PID of the current shell
\$!	PID of the most recent background pipeline

```
$ ./arg.sh abc
$0 is ./arg.sh
$1 is abc
$2 is
$# is 1
```

```
./my_web_server &
echo $! > /var/run/my_server.pid
```

Remember Positional arguments start at \$1. \$0 is the script name.

"\$@" vs "\$*"

```
# Suppose the script receives three arguments:
#   one  "two words" three

for x in "$@";
do printf '<%s>\n' "$x";
done
# <one>
# <two words>
# <three>

printf '<%s>\n' "$*"
# <one two words three>
```

Preferred for forwarding arguments Use "\$@" when you want to preserve the original argument boundaries.

3. Tests & Control Flow

Exit status, test expressions, arithmetic, conditions, and loops.

Exit Status: The Shell's Boolean

- Every command returns an integer exit status.
- **0 means success**; nonzero means failure or another condition.
- **\$?** contains the status of the most recently executed command.
- **Control structures**, **&&** and **||** make decisions from exit status.

```
grep -q '^root:' /etc/passwd  
printf 'status = %s\n' "$?"
```

The test Command and [...]

```
test -f /etc/passwd  
printf '%s\n' "$?"
```

```
[ -f /etc/passwd ]  
printf '%s\n' "$?"
```

- [is a command; the closing] is a required argument.
- Whitespace around [, operators, and] is required.
- Quote variable expansions inside tests.

Example ["\$mode" = production] is safe even when \$mode is empty.

File Tests

Test	True when...
-e file	Path exists
-f file	Regular file
-d file	Directory
-L file	Symbolic link
-r / -w / -x	Readable / writable / executable
-s file	Exists and size is greater than zero
file1 -nt file2	file1 is newer than file2

String Tests

Expression	Meaning
-z "\$s"	String length is zero
-n "\$s"	String length is nonzero
"\$a" = "\$b"	Strings are equal
"\$a" != "\$b"	Strings are different

Portability Use = rather than == in POSIX sh. Avoid < and > for string ordering inside [...].

Integer Comparisons

Operator	Meaning
-eq	equal
-ne	not equal
-gt	greater than
-ge	greater than or equal
-lt	less than
-le	less than or equal

```
if [ "$count" -gt 10 ]; then
    printf 'large\n'
fi
```

Combine Conditions with the Shell

```
[ -f "$cfg" ] && printf 'config exists\n'
[ -d "$dir" ] || mkdir -p "$dir"

if [ -r "$file" ] && [ -s "$file" ]; then
    process "$file"
fi
```

- && runs the right side only if the left side succeeded.
- || runs the right side only if the left side failed.
- Prefer this over -a and -o inside [...], which are historically ambiguous.

Arithmetic Expansion

```
count=8
count=$((count + 9))
count=$((count * 2))
printf '%s\n' "$count"
```

- `$(( ... ))` performs integer arithmetic in the shell.
- Inside arithmetic expansion, variable names usually do not need `$`.
- Arithmetic expansion is clearer than invoking **expr** for simple integer math.

if / elif / else

```
if [ "$mode" = production ]; then
    start_prod
elif [ "$mode" = test ]; then
    start_test
else
    printf 'Unknown mode: %s\n' "$mode" >&2
    exit 2
fi
```

- The branch is selected by exit status, not by a built-in Boolean type.

case: Pattern-Based Selection

```
case "$answer" in
    [Yy] | [Yy][Ee][Ss])
        printf 'yes\n'
        ;;
    [Nn] | [Nn][Oo])
        printf 'no\n'
        ;;
    *)
        printf 'invalid input\n' >&2
        ;;
esac
```

- Patterns use shell glob syntax, not regular expressions.
- Each branch ends with `;;` in portable sh.

for Loops

Values explicitly

- `for x in red green blue; do ...; done`
- Useful for small known sets
- Quote "\$x" when using it

Files safely

- `for f in *.log; do ...; done`
- Do not parse `ls` output in a loop
- Handle the no-match case when needed

Avoid `for f in $(ls)` breaks on spaces, tabs, newlines, and glob characters in filenames.

while and read

```
while IFS= read -r line; do
    printf 'line: %s\n' "$line"
done < input.txt
```



```
count=0
cat data.txt | while read -r line;
do count=$((count + 1))
done
echo "Total lines: $count" # always 0
```



- IFS= preserves leading/trailing whitespace.
- read -r prevents backslash interpretation.
- Redirect the loop input instead of piping into it when you need variable changes after the loop.

Read Interactive Input Safely

```
printf "Delete temporary files? [y/N] "  
IFS= read -r answer  
  
case "$answer" in  
    [Yy]|[Yy][Ee][Ss])  
        rm -f -- "$tmpfile"  
        ;;  
    *)  
        printf 'Cancelled\n'  
        ;;  
esac
```

- Use printf instead of echo -n for portable prompts.
- Default to the safer action when input is unexpected.
- Use -- before filenames when a command supports it.

Temporary Files and Directories

```
tmpdir=$(mktemp -d) || exit 1
trap 'rm -rf "$tmpdir"' EXIT HUP INT TERM

tmpfile=$tmpdir/output.txt
printf 'program output\n' > "$tmpfile"
```

- Let mktemp choose a unique name.
- Keep temporary files inside one private temporary directory.
- Register cleanup immediately after creation.

4. Functions & Options

Structure scripts, validate input, and parse command-line options.

Functions

```
log() {  
    printf '[%s] %s\n' "$(date +%H:%M:%S)" "$*"  
}  
  
log 'service started'
```

- Define a function before it is first used.
- A function has its own positional parameters (\$1, \$2, ...).
- The function shares shell variables unless the shell provides a non-POSIX local mechanism.

Scope and Subshells

Same shell

- Function assignments can modify caller variables
- Environment variables are just shell variables marked for export
- Changes persist after the function returns

Subshell / child process

- Receives a copy of exported variables
- Changes do not propagate back to the parent

Portable design Do not depend on variable changes made inside a pipeline being visible afterward.

Function Arguments and Return Status

```
copy_if_readable() {  
    [ "$#" -eq 2 ] || return 2  
    [ -r "$1" ] || return 3  
    cp -- "$1" "$2"  
}  
  
copy_if_readable source.txt backup.txt  
status=$?
```

- return sets an exit status (0–255); it is not a general-purpose return value.
- Use stdout to return textual data from a function.
- Validate the number of arguments before using \$1, \$2, etc.

Parsing Options with getopt

```
verbose=0
file=
while getopt 'vf:' opt; do
    case "$opt" in
        v) verbose=1 ;;
        f) file=$OPTARG ;;
        ?) exit 2 ;;
    esac
done
shift $((OPTIND - 1))
```

- A colon after an option letter means that option requires an argument.
- \$OPTARG contains the current option argument.
- \$OPTIND points to the next argument to process.
- After getopt, shift away the parsed options.

5. Robust Scripts

Errors, cleanup, signals, tracing, and static analysis.

Error Handling Starts with Exit Status

- Check failures that matter instead of assuming every command succeeded.
- Send diagnostics to stderr: `printf ... >&2`.
- Use meaningful nonzero exit codes for invalid input or failed operations.
- Do not hide command failures with long pipelines or unchecked command substitutions.

```
if ! cp -- "$src" "$dst"; then
    printf 'copy failed: %s -> %s\n' "$src" "$dst" >&2
    exit 1
fi
```

```
./your_script.sh > output.log 2> error.log
```

Shell Options: Useful, but Understand Them

Option	Effect	Caution
set -u	Unset parameter expansion becomes an error	Use defaults for optional variables
set -x	Trace commands after expansion	May expose secrets in logs

```
set -u
APP_PORT=${PORT}
echo "Port: $APP_PORT"
```

./unbound.sh: line 4: **PORT: unbound variable**

```
set -u
APP_PORT=${PORT:-8080}
echo "Port: $APP_PORT"
```

Port: 8080

```
#!/bin/bash
set -x
USER_NAME="Alice"
echo "Hello, $USER_NAME"
```

+ **USER_NAME=Alice**
+ **echo 'Hello, Alice'**
Hello, Alice

Course recommendation Use explicit checks for important operations. Add shell options deliberately, not as magic safety switches.

trap: Cleanup and Signals

```
tmpdir=$(mktemp -d) || exit 1

cleanup() {
    rm -rf "$tmpdir"
}

trap cleanup EXIT HUP INT TERM
```

- EXIT is useful for normal cleanup at script termination.
- Use signal names for readability instead of numeric values.
- SIGKILL and SIGSTOP cannot be caught or ignored.

Signals You Commonly Meet

Signal	Typical meaning	Catchable?
SIGHUP	Terminal/session ended or reload convention	Yes
SIGINT	Interrupt from Ctrl+C	Yes
SIGQUIT	Quit from terminal	Yes
SIGTERM	Polite termination request	Yes
SIGKILL	Immediate termination	No
SIGSTOP	Stop process	No

Debugging with Tracing

```
$ sh -x script.sh
+ config=/etc/app.conf
+ '[' -r /etc/app.conf ']'
+ start_service
...
```

- `sh -x` prints commands after expansions and before execution.
- You can enable tracing inside a script with `set -x` and disable it with `set +x`.
- Be careful: tracing may print passwords, tokens, or other secrets.

ShellCheck

- ShellCheck is a static analyzer for shell scripts.
- It catches common quoting mistakes, suspicious expansions, portability issues, and unreachable constructs.
- Use it early:
`shellcheck script.sh`
- Treat warnings as explanations to investigate, not as blind rewrite instructions.

Workflow Edit → shellcheck → test with representative inputs → review exit status and logs.

6. Regular Expressions

Pattern matching for grep, sed, awk, and other text-processing tools.

Regular Expressions: The Core Idea

- A regular expression describes **a set of strings**.
- **Concatenation** places patterns in sequence.
- **Alternation** chooses between patterns.
- **Repetition** matches a pattern multiple times.
- Do not confuse regular expressions with shell glob patterns such as *.log.

Two different pattern languages Shell globbing chooses **pathnames**; regular expressions usually match **text inside data**.

Common ERE Operators

Operator	Meaning	Example
.	Any single character	r.n
[abc]	One listed character	[Yy]
[^abc]	One character not listed	[^0-9]
^ / \$	Start / end of line	^root:
*	Zero or more	ab*
+	One or more	[0-9]+
?	Zero or one	colou?r
{m,n}	Repetition range	[0-9]{2,4}

Regex Examples

```
^John
```

```
# lines starting with John
```

```
[Ee][Nn][Dd]$
```

```
# lines ending in end, End, END, ...
```

```
^[A-Za-z0-9_]+$
```

```
# one or more letters, digits, or underscores
```

Read patterns left to right Anchors constrain position; character classes constrain one character; repetition modifies the preceding atom.

BRE, ERE, and Tool Differences

Tool	Default regex	Extended form
grep	BRE	grep -E
sed	BRE	sed -E (common on modern BSD/GNU systems)
awk	ERE	ERE is built in

Practical advice When teaching or debugging a pattern, always state which tool and regex dialect you are using.

7. sed & awk

Two classic tools for selecting, transforming, and summarizing text streams.

sed: Stream Editor

- Reads input a line at a time.
- Applies editing commands to the current pattern space.
- Prints the result to stdout unless **-n** suppresses automatic output.
- Common tasks: select lines, delete lines, substitute text, and insert/append content.

```
sed -e '1,10d' -e 's/yellow/black/g' input.txt
```

sed Addresses

Address	Meaning
10	Line 10
/error/	Lines matching the regex
10,20	Lines 10 through 20
10,/END/	From line 10 through the next line matching END

```
sed '10,20d' file.txt  
sed '/^#/d' file.txt
```

sed: Print and Delete

Print selected lines

- `sed -n '/^root:/p' /etc/passwd`
- `-n` suppresses normal automatic output
- `p` prints only the selected lines

Delete selected lines

- `sed '/^#/d' file`
- `d` removes matching lines from the stream
- **Original file is unchanged** unless you explicitly use in-place editing

sed: Substitution

```
sed 's/old/new/' file.txt.      # first match per line
sed 's/old/new/g' file.txt      # all matches
sed -n 's/error/ERROR/p' log.txt # print changed lines only
```

Flag	Meaning
g	Replace all matches on the line
p	Print the line when substitution succeeds
N	Replace only the Nth occurrence

sed: Insert, Append, and Change

```
# sed script file
/^ERROR/i\
--- important event follows ---

/^END/a\
--- section complete ---
```

- i inserts before the selected line.
- a appends after the selected line.
- c replaces the selected line.
- For complex transformations, keep the sed program in a separate file.

awk: Pattern → Action

```
awk 'pattern { action }' data.txt
```

```
awk '{ print $1 }' data.txt
```

```
awk '/ERROR/ { print $1, $3 }' log.txt
```

- Input is split into records (normally lines).
- Each record is split into fields.
- A missing pattern means “every record.”
- A missing action means “print the record.”

awk Patterns

Pattern	When it runs
/regex/	Current record matches the regex
BEGIN	Before any input is read
END	After all input is processed
expression	Expression evaluates as true

```
awk 'BEGIN { print "start" }  
    /error/ { count++ }  
    END { print count }' log.txt
```

```
awk '$3 > 1000 {print $1}' file.txt
```

awk Built-in Variables

Variable	Meaning
\$0	Entire current record
\$1, \$2, ...	Fields in the current record
NF	Number of fields in the current record
NR	Record number
FILENAME	Current input filename
FS	Input field separator
OFS	Output field separator

awk Example: /etc/passwd

```
awk -F: '$1 == "root" {  
    print "uid=" $3, "home=" $6, "shell=" $7  
}' /etc/passwd
```

- -F: sets the field separator to a colon.
- \$1 is the username, \$3 is UID, \$6 is home directory, and \$7 is login shell.
- **awk is often clearer** than a long chain of **grep + cut** when selection and field logic belong together.

8. Case Study

Turn a fragile monitoring script into a small, readable, testable program.

Case Study: Checking Whether a Host Responds

Fragile approach

- Parse human-readable ping statistics
- Write predictable temporary filenames
- Leave variables unquoted
- Hard-code mail parsing around one ping implementation

Better approach

- Use ping exit status directly
- Validate exactly one host argument
- Quote all expansions
- Return a meaningful status and let a scheduler decide what to do

A Simpler Health-Check Script

```
#!/bin/sh
usage() {
    printf 'Usage: %s HOST\n' "$0" >&2
}
[ "$#" -eq 1 ] || { usage; exit 2; }
host=$1
if ping -c 3 "$host" >/dev/null 2>&1; then
    printf '%s is reachable\n' "$host"
    exit 0
else
    printf '%s is unreachable\n' "$host" >&2
    exit 1
fi
```

- No temporary file is required. No localization-sensitive output parsing is required.

- A scheduler or monitoring system can use the exit status. Add notifications separately so the health check stays testable.

Shell Scripting Checklist

- Choose the intended interpreter and write to that language.
- Quote expansions by default; use "\$@" for argument forwarding.
- Check exit status; make errors visible on stderr.
- Prefer safe loops and input handling over parsing ls or echo quirks.
- Clean up temporary resources with trap.
- Use ShellCheck, then test real edge cases.
- Use sed and awk when they make text-processing logic clearer—not merely shorter.

References

- POSIX Shell Command Language (The Open Group)
- `sh(1)`, `test(1)`, `sed(1)`, `awk(1)`, `getopts(1)` on your target system
- GNU Bash manual — useful when you intentionally use Bash extensions
- ShellCheck — static analysis for shell scripts
- Debian and FreeBSD manual pages for implementation-specific behavior

Course principle Check the manual page on the system you are actually administering.